

Year Up United Data Analyst Academy

Pre-Work Week 1 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

Table of Contents

MODULE 1 FILE SYSTEMS & THE COMMAND LINE3

 EXERCISE 1.A BUILDING YOUR ACADEMY WORKSPACE..... 4

 EXERCISE 1.B NAVIGATING WITH INTENTION..... 6

 EXERCISE 1.C NAVIGATION WITHOUT RESETTNG 8

MODULE 2 INTRODUCTION TO VERSION CONTROL WITH GIT10

 EXERCISE 2.A INITIALIZE A REPOSITORY AND CREATE A REMOTE 11

 EXERCISE 2.B ADD, COMMIT, AND PUSH EXISTING FILES 13

 EXERCISE 2.C CREATE MEANINGFUL, SEPARATE COMMITS 15

Module 1

File Systems & The Command Line

Exercise 1.A Building Your Academy Workspace

This folder structure will be used throughout the academy.

Consistency matters.

Scenario

You are preparing your workspace for a 12-week data analytics academy.

Your folders should:

- Be clearly organized
- Follow consistent naming conventions
- Be easy to navigate using Git Bash

Part 1 – Create the Main Project Folder

Using Git Bash:

1. Navigate to your `Documents` folder
2. Using the `mkdir` command, create a folder named `DataAnalytics`
3. Navigate into the `DataAnalytics` folder
4. Confirm your location using `pwd`

Part 2 – Create Pre-Work Week Folders

Inside `DataAnalytics`, create the following folders:

```
pre-week-1  
pre-week-2  
pre-week-3
```

These represent the three pre-work weeks.

Part 3 – Create Main Academy Week Folders

Still inside `DataAnalytics`, create folders for:

```
week-01  
week-02  
week-03  
...  
week-12
```

Exercise 1.B Navigating with Intention

This exercise focuses on moving confidently through your directory structure using Git Bash.

No File Explorer.

Scenario

You are beginning work for Pre-Week 1.

Before adding files, you need to confirm that you can move through your academy folder structure efficiently.

Starting Point

Navigate to your `DataAnalytics` folder.

Confirm your location using `pwd`.

Part 1 – Enter Pre-Week 1

1. Navigate into `pre-week-1`
2. Confirm your location using `pwd`
3. Create the following folders inside `pre-week-1`:

```
raw-data  
notes  
assignments
```

4. Use `ls` to confirm your work

Part 2 – Move Between Subfolders

Starting inside `pre-week-1`:

1. Navigate into `raw-data`

2. Create a file named:

```
sample-data.csv
```

3. Confirm it exists using `ls`

4. Move back to `pre-week-1`

5. Navigate into `notes`

6. Create a file named:

```
terminal-practice.txt
```

7. Confirm it exists using `ls`

8. Navigate back to `pre-week-1`

Exercise 1.C Navigation Without Resetting

This exercise focuses on moving through your directory structure intentionally and efficiently.

No File Explorer.

Do not navigate back to your home directory unless instructed.

Scenario

You are preparing to work across multiple weeks of the academy.

You must demonstrate that you can move between week folders confidently using relative paths.

Starting Point

Navigate to your `DataAnalytics` folder.

From there, navigate into `pre-week-1/notes`

Confirm your location using `pwd`.

Part 1 – Cross-Week Navigation

Without returning to `DataAnalytics` first:

1. Navigate to `pre-week-2`
2. Confirm your location
3. Navigate to `week-01`
4. Confirm your location

Part 2 – Deep Navigation

Starting inside `week-01`:

1. Navigate **directly** to `pre-week-1/raw-data` using a relative path
2. Confirm your location
3. Return to `DataAnalytics`

Part 3 – Verification

While inside `DataAnalytics`:

1. Use `ls` to verify all folders are present
2. Confirm that:

```
pre-week-1
pre-week-2
pre-week-3
week-01
...
week-12
```

are all visible

Submitting Your Work

1. Take a screenshot of your Git Bash terminal showing the output of your `ls` command
2. Upload this screenshot to Brightspace under **Pre-Work Week 1 Homework 1**.

Module 2

Introduction to Version Control with Git

Exercise 2.A Initialize a Repository and Create a Remote

Scenario

You have created your academy folder structure.

Next you need to place it under version control to keep your work tracked and backed up.

Starting Point

Navigate to your `DataAnalytics` folder.

Confirm your location using `pwd`.

Part 1 – Initialize the Local Repository

1. Initialize Git:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics
$ git init
Initialized empty git repository in c:/Users/timmk/Documents/DataAnalytics/.git/
```

2. Confirm Git is active using `git status`:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git status
On branch main

No commits yet

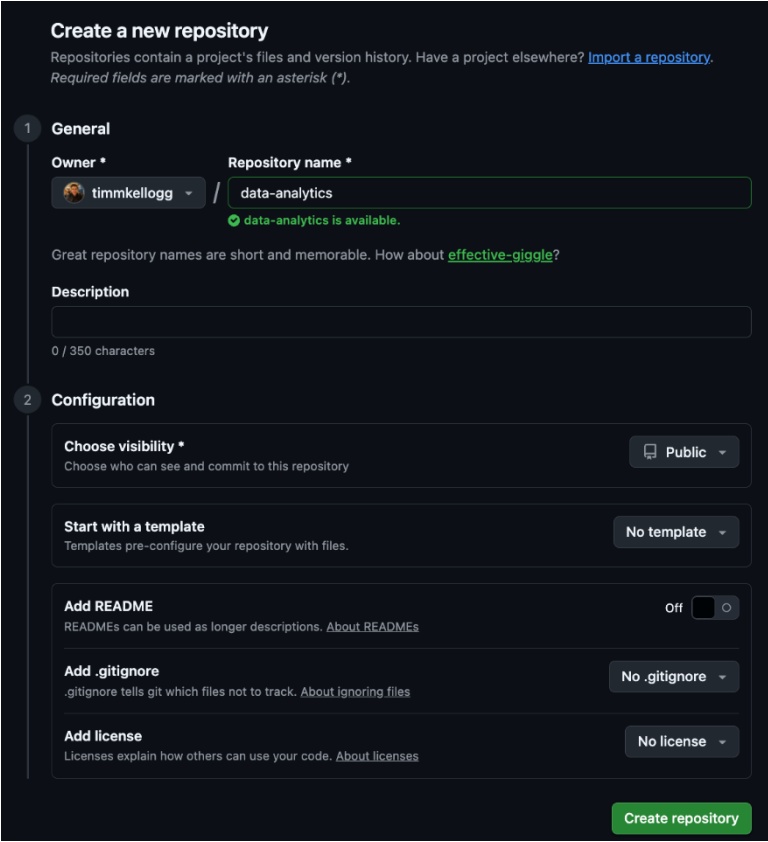
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  pre-week-1/

nothing added to commit but untracked files present (use "git add" to track)
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$
```

Note: Git ignores empty directories. You will only be able to stage and commit folders that at least one file inside.

Part 2 – Create the Remote Repository

1. Create a new repository on GitHub called `data-analytics`



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * timmkello / **Repository name *** data-analytics
data-analytics is available.

Great repository names are short and memorable. How about [effective-giggle](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Start with a template No template

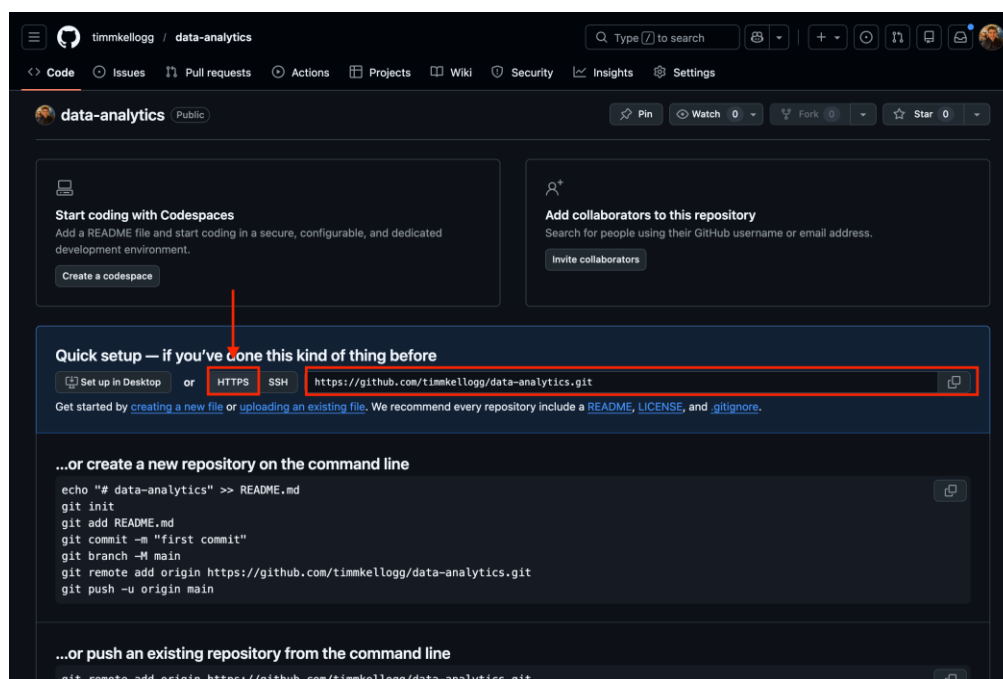
Add README Off

Add .gitignore No .gitignore

Add license No license

Create repository

2. Copy the repository URL for use in the next exercise



Exercise 2.B Add, Commit, and Push Existing Files

Scenario

You have initialized your local Git repository and created a remote repository in GitHub. Next you need to connect the local repo to the remote to begin staging and committing your files.

Part 1 – Connect Local to Remote

1. Inside `DataAnalytics`, connect your repository to GitHub:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git remote add origin https://github.com/timmkellogg/data-analytics.git
```

2. Verify the connection using `git remote -v`:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git remote -v
origin https://github.com/timmkellogg/data-analytics.git (fetch)
origin https://github.com/timmkellogg/data-analytics.git (push)
```

Part 2 – Stage and Commit

1. Stage all existing files and folders:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git add .
```

2. Create a commit with a clear message

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git commit -m 'add pre-work files and structure'
```

Part 3 – Push to Remote

1. Push your commit:

```
timmk@TIMKELLOGG3D88 CLANGARM64 ~/Documents/DataAnalytics (main)
$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (5/5), 330 bytes | 330.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/timmkellogg/data-analytics.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

2. After pushing, confirm your files appear on GitHub.

Exercise 2.C Create Meaningful, Separate Commits

Scenario

In professional environments, commits represent logical units of work.

Instead of committing everything at once, you will practice creating focused commits with distinct messages.

All work in these exercises takes place inside `DataAnalytics/pre-week-1`.

Starting Point

Navigate to your `DataAnalytics/pre-week-1` folder.

Confirm your location using `pwd`.

Part 1 – Create New Files

1. Create the following files:

```
notes.md
questions.txt
stats.csv
```

Part 2 – Stage and Commit Separately

For each file:

1. Stage only that file
2. Create a commit with a clear, specific message

Each file must have its own commit.

Part 3 – Push

1. After all commits are completed, push your changes to GitHub.

Submitting Your Work

1. Copy your remote repo URL and submit to Brightspace under **Pre-Work Week 1 Homework 2**.

Note:

Git does not track empty directories.

If a folder contains no files, it will not appear in the remote repository (yet).